

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA5117

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4, BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

I Harshini R N (192421175) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Harshini R N

Annexure A

Case Study

1. A software company is using MD5 to store user passwords in its database. After a security breach, it is discovered that attackers were able to crack multiple passwords. Analyze the weaknesses of MD5 in this scenario and propose a more secure hashing approach using SHA-based techniques.
2. A banking application implements digital signatures for transaction verification, but users report unauthorized transactions being approved. Analyze the possible flaws in the digital signature implementation and recommend improvements based on secure signature standards.
3. An enterprise network uses Kerberos for authentication, but attackers successfully perform a replay attack to gain unauthorized access. Examine how this attack could occur and propose enhancements in the authentication process to prevent such threats.
4. A secure communication system uses a simple hash function to verify message integrity, but attackers are able to modify messages without detection. Analyze why this approach fails and propose a solution using HMAC to improve message authentication.
5. An organization implements X.509 certificates for secure communication, but users encounter frequent trust validation errors. Analyze the possible issues in certificate management and propose solutions to ensure proper authentication and trust chain validation.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1. MD5 for Password Storage: Weaknesses and SHA-based Solutions

Analysis of MD5 Weaknesses:

The use of MD5 for password storage is a critical security flaw for several reasons:

- **Computation Speed:** MD5 was designed to be computationally fast. In a password-cracking scenario, this is a massive weakness. Attackers using modern GPUs or ASICs can compute billions of MD5 hashes per second, making brute-force and dictionary attacks highly effective.
- **Lack of Built-in Salting:** If the company used raw MD5 without salting (appending random data to each password before hashing), attackers can use precomputed "Rainbow Tables" to instantly reverse the hashes into plaintext passwords.
- **Collision Vulnerabilities:** While less critical for password cracking than for digital signatures, MD5 is mathematically broken. Multiple different inputs can produce the exact same MD5 hash, weakening its overall cryptographic reliability.

Proposed SHA-Based Solution:

A secure password hashing mechanism must be slow (computationally expensive) and include a unique salt for every user.

- **Recommendation:** Use **PBKDF2 (Password-Based Key Derivation Function 2) combined with HMAC-SHA256** (or SHA-512).
- **How it works:** PBKDF2 applies a cryptographic hash function (like SHA-256) to the password and a unique, random salt, and then iterates this process thousands of times (e.g., 210,000+ iterations). This deliberate slowing down makes brute-force and dictionary attacks economically unfeasible for attackers, while the unique salt completely neutralizes Rainbow Table attacks.

2. Flaws in Digital Signatures for Banking Applications

Analysis of Possible Flaws:

If unauthorized transactions are being approved despite the use of digital signatures, the implementation is fundamentally flawed. Possible reasons include:

- **Replay Attacks (Lack of Message Binding):** The signature might be cryptographically valid, but it is not bound to a specific context. An attacker might intercept a legitimate signed transaction (e.g., "Transfer \$100") and resend it multiple times. If the signature doesn't include a unique identifier or timestamp, the system processes it again.
- **Weak Hashing Algorithms:** The application might be using outdated hashing algorithms (like MD5 or SHA-1) before signing. Attackers could perform a collision attack to alter the transaction details without invalidating the signature.
- **Poor Key Management:** The private keys of the users might be stored insecurely on their devices (e.g., in plaintext in the app's local storage), allowing malware to steal them and forge signatures.

- **Flawed Random Number Generation (RNG):** Algorithms like ECDSA require a secure random number (nonce) for every signature. If the RNG is predictable, attackers can reverse-engineer the user's private key from public signatures.

Recommended Improvements:

- **Implement Nonces and Timestamps:** Include a cryptographic nonce (a random number used once) and a timestamp within the transaction payload *before* signing. This ensures every signature is unique, completely preventing replay attacks.
- **Upgrade Cryptographic Standards:** Use strictly secure algorithms, such as **RSA-PSS** with a minimum 2048-bit key length, or **ECDSA** (Elliptic Curve) combined with **SHA-256** or higher.
- **Secure Key Enclaves:** Ensure private keys are generated and stored inside the device's hardware-backed Secure Enclave or Trusted Execution Environment (TEE), making them inaccessible to malware.

3. Kerberos Authentication and Replay Attacks

Analysis of the Attack:

Kerberos relies heavily on time-stamped "Authenticators" to prevent replay attacks. If an attacker successfully performed a replay attack, one or more of the following failures occurred:

- **Excessive Time Skew Window:** Kerberos allows a certain time difference (skew) between the client and server to account for network delays (default is often 5 minutes). If this window is configured to be too large, an attacker has ample time to intercept a valid ticket and authenticator and reuse it.
- **Disabled Replay Cache:** Within the allowable time skew window, the server is supposed to keep a "Replay Cache" of recently seen authenticators. If an attacker resends an authenticator that is still within the 5-minute window, the cache detects the duplicate and drops it. If this cache is disabled, misconfigured, or running out of memory, replay attacks will succeed.
- **Compromised Time Synchronization:** An attacker might have compromised the network's NTP (Network Time Protocol) server, rolling back the clock on the Application Server to trick it into accepting expired authenticators.

Proposed Enhancements:

- **Enforce Strict Replay Caching:** Ensure the application server's Kerberos implementation has a fully functioning, persistent replay cache that strictly tracks all authenticators received within the allowable time window.
- **Tighten Time Skew:** Reduce the acceptable time skew from 5 minutes to a much tighter window (e.g., 60 seconds) to minimize the attack surface.
- **Secure NTP (NTS):** Implement Network Time Security (NTS) to cryptographically verify time synchronization across the domain, preventing attackers from manipulating server clocks.

4. Simple Hash Function vs. HMAC for Message Integrity

Analysis of the Failure:

A simple hash function (e.g., $\text{Hash}(\text{Message})$) only provides integrity against *accidental* corruption, not malicious alteration.

- **No Authentication:** If the system just sends the message and its hash, an attacker can simply intercept the message, modify it, compute a brand new hash for the modified message, and send both to the receiver. The receiver will calculate the hash, see a match, and assume the message is authentic.
- **Length Extension Attacks:** Even if a secret key is concatenated with the message in a naive way (e.g., $\text{Hash}(\text{Key} + \text{Message})$), it is vulnerable to a Length Extension Attack. An attacker who intercepts the hash and knows the length of the key can mathematically append malicious data to the message and calculate a valid new hash without ever knowing the secret key.

Proposed Solution (HMAC):

- **Implement HMAC (Hash-based Message Authentication Code):** HMAC solves this by securely mixing a secret cryptographic key with the message using a two-pass hashing mechanism: $\text{HMAC}(K, m) = H((K \oplus \text{opad}) || H((K \oplus \text{ipad}) || m))$.
- **Why it works:** Because the attacker does not possess the secret key (K), they cannot generate a valid HMAC tag for their modified message. Furthermore, the inner and outer padding (*ipad* and *opad*) mathematically destroy the vulnerabilities that allow Length Extension Attacks. HMAC guarantees both the **integrity** of the data and the **authenticity** of the sender.

5. X.509 Certificate Validation and Trust Chain Errors

Analysis of Possible Issues:

Frequent trust validation errors in an X.509 Public Key Infrastructure (PKI) usually stem from configuration or lifecycle management failures rather than cryptographic breaks:

- **Broken Trust Chain (Missing Intermediates):** Servers must send their own certificate *and* any intermediate certificates required to link back to a Root CA. If the server only sends its leaf certificate, the client's browser cannot build the chain of trust and will throw an error.
- **Untrusted Root CA:** If the organization is using an internal/private Certificate Authority (CA) to issue certificates, but the clients' machines have not been configured to trust this internal Root CA, all certificates will be rejected.
- **Certificate Expiration:** Poor lifecycle management leads to certificates quietly expiring, immediately triggering validation errors for all users.
- **Hostname/SAN Mismatch:** The URL the user is trying to reach (e.g., `[https://api.internal.com](https://api.internal.com)`) does not exactly match the Common Name (CN) or Subject Alternative Name (SAN) listed on the certificate.

- **Revocation Check Failures:** Clients may be trying to check if the certificate is revoked via OCSP or CRL, but restrictive firewalls block access to the CA's revocation servers, causing a "soft fail" or "hard fail" error depending on the client.

Proposed Solutions:

- **Bundle Intermediate Certificates:** Configure all web servers and load balancers to serve the complete certificate chain (leaf + intermediates) so clients can seamlessly verify the path to the Root CA.
- **Automate Lifecycle Management:** Implement the ACME protocol (e.g., Let's Encrypt or internal tools like HashiCorp Vault) to automatically renew and deploy certificates 30 days before expiration.
- **Centralized Trust Management:** If using an internal CA, use Group Policy Objects (GPO) or Mobile Device Management (MDM) to seamlessly push the internal Root CA certificate to the trusted root store of all employee devices.
- **Implement OCSP Stapling:** Instead of forcing clients to query the CA for revocation status, configure the server to periodically fetch a time-stamped OCSP response and "staple" it to the initial TLS handshake, improving both privacy and reliability.